

ICMC-USP

Notas de Aula
SME0305, SME0306 & SME0602

Métodos Numéricos e Computacionais

com Aplicações para Engenharia e Programação em python

Roberto F. Ausas

April 15, 2024

Instituto de Ciências Matemáticas e de Computação
Universidade de São Paulo

Prefacio

Este documento está organizado por capítulos, cada um dos quais apresentando os principais métodos do cálculo numérico e computacional para os cursos de engenharia. Para cada tema, se apresentam os principais conceitos teóricos por trás dos métodos numéricos, algumas aplicações de interesse para a engenharia, a sua implementação computacional e, finalmente, problemas e atividades que o aluno precisará desenvolver, com as quais, este será avaliado ao longo do curso.

Que tópicos este curso apresenta

Nas disciplinas de cálculo numérico para engenharia são estudados os seguintes temas, que temos dividido em capítulos (a depender do curso específico, alguns tópicos poderão não estar inclusos):

- Capítulo |01|** Introdução à programação em python;
- Capítulo |02|** Redes em python: Sistemas Lineares 1;
- Capítulo |03|** Problema de Calor e Iterações na Engenharia: Sistemas Lineares 2;
- Capítulo |04|** Sistemas Sobredeterminados e Analise de Dados (sme0305);
- Capítulo |04|** Aproximação de Funções (sme0306 e sme0602);
- Capítulo |05|** Aproximação de Autovalores e Autovetores;
- Capítulo |06|** Resolução Numérica de EDOs (sme0306 e sme0602);

RFA

Contents

Prefacio	ii
Contents	iii
1 PROGRAMAÇÃO EM python	1
1.1 Preludio	1
1.2 Lista 1 de exercícios	2
2 CÁLCULO DE REDES EM python: Resolução de Sistemas Lineares 1	5
2.1 Preludio	5
2.1.1 Exemplo de resolução de uma rede hidráulica	5
2.1.2 Conexão da rede	7
2.2 Redes em python	12
2.3 Lista 2 de exercícios	15
2.4 Cálculos Monte Carlo em Redes	16
2.5 Atividade 1	18
2.5.1 Exercícios opcionais	19
3 PROBLEMA DE CALOR e ITERAÇÕES NA ENGENHARIA: Resolução de Sistemas Lineares 2	21
3.1 Preludio	21
3.2 Transferência de Calor por Condução	21
3.3 Métodos iterativos	24
3.4 Lista 3 de exercícios	27
Alphabetical Index	29

1.1 Preludio

1.1 Preludio 1

1.2 [Lista 1 de exercícios](#) 2

Nas primeiras aulas precisamos introduzir a principal ferramenta de trabalho que será utilizada nas disciplinas de Métodos Numéricos e Computacionais. Depois de tudo, a ideia do cálculo numérico é resolver problemas complexos de Engenharia, que não poderiam ser resolvidos manualmente, sem o auxílio de um computador.

De maneira muito geral, as linguagens de programação, podem ser divididas em duas categorias:

- **Linguagens compiladas:** Dentre as primeiras temos linguagens tais como C, C++ e Fortran. A escrita de código neste tipo de linguagens de programação requer um domínio maior da sintaxe e das funcionalidades da linguagem. Como contrapartida, este tipo de linguagens produzem códigos que são muito rápidos pois tem sido otimizadas ao longo dos anos, e por tanto são usadas amplamente na Engenharia. De fato, a maioria dos códigos de cálculo usados na indústria (*Open Source* ou comerciais) estão feitos com elas. Ao **compilar** o código e gerar um arquivo binário otimizado, é possível tirar o maior proveito do poder de processamento de um computador.
- **Linguagens interpretadas:** Por outra parte, as linguagens interpretadas, como python e Mat Lab, são relativamente simples e intuitivas, pela sua flexibilidade na sintaxe, tornando o processo de desenvolvimento mais rápido e ágil. De maneira grosseira, o código vai sendo executado a medida que se **interpreta**. Porém, se não são tomados alguns recaudos no desenho do código, a performance computacional delas pode estar bastante aquém do necessário para resolver problemas de grande porte. Um exemplo prototípico disto, é quando utilizamos uma estrutura de repetição (como um `for`) no qual realizamos um grande número de operações em cada passo. Ao longo de curso iremos chamando a atenção sobre isto, tentando introduzir boas práticas de programação.

Como comparação, vejamos o exemplo de um código simples para criar um array com números de ponto flutuante de dupla precisão, popular ele com os números $0, \dots, N$ e imprimir o resultado na tela, usando a linguagem compilada C (acima) e a linguagem interpretada python (embaixo).

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    int i, N = 10;
    double *array;
    array=(double *) malloc(N*sizeof(double));
    for(i=0; i < N; i++) {
        array[i] = (double) i;
        printf("%lf\n", array[i]);
    }
    free(array);
    return 0;
}
```

```
import numpy as np
N = 10
array = np.arange(N)
print(array)
```

Olhando para o exemplo, já vemos que uma linguagem interpretada como python se torna mais prática para um curso de cálculo numérico, pois o objetivo é entender os métodos numéricos, os algoritmos e como resolver problemas práticos no computador, sem gastar muito tempo no desenvolvimento de código.

Neste curso iremos adotar a linguagem python, a qual tem-se tornado bastante popular nos últimos anos. Para facilitar a implementação dos diferentes métodos numéricos, contamos com algumas bibliotecas específicas que facilitarão o trabalho:

- ▶ **numpy**: Para manipulação eficiente de matrizes e vetores, operações de álgebra linear computacional e vários métodos numéricos.
- ▶ **scipy**: Para métodos numéricos mais avançados ou específicos, não cobertos pela anterior.
- ▶ **matplotlib**: Para plotagens e geração de gráficos em 2D e 3D.

Aviso importante

O material de estudo para este capítulo serão os jupyter notebooks desenvolvidos pelo professor na sala de aula, que foram disponibilizadas no repositório da disciplina. Outras fontes de informação a ser consideradas são os sites das bibliotecas que iremos utilizar, tais como <https://numpy.org> e <https://matplotlib.org/>, assim como consultas dirigidas ao professor em forma presencial ou por e-mail (rfausas@icmc.usp.br).

1.2 Lista 1 de exercícios

A lista de exercícios a ser apresentada em avaliação oral por sorteio é dada na sequência:

Exercícios

1. Fazer em python uma função que:

- Pega dois vetores randômicos $\mathbf{a}$ e $\mathbf{b}$ de dimensão n , e dois escalares randômicos α e β e calcula um vetor $\mathbf{c}$ tal que

$$\mathbf{c} = \alpha \mathbf{a} + \beta \mathbf{b}$$

- Pega uma matriz randômica $\mathbf{A}$ de $n \times n$ e calcula a sua m -essima potência

$$\mathbf{A}^m = \underbrace{\mathbf{A} \dots \mathbf{A}}_{m \text{ vezes}}$$

(tomar valores de $m = 2, 3, 4$).

Em todos os casos medir o tempo necessário para realizar as operações para diferentes dimensões n . Plotar o tempo de cálculo como função da dimensão n usando a escala linear padrão e a escala $\log \log$. No segundo ponto, colocar no mesmo gráfico os resultados para os diferentes valores de m . Tirar conclusões.

Nota: Para que os resultados sejam interessantes, no primeiro ponto, tomar valores de $n = 10^5, 10^6, \dots, 10^9$. Já, no segundo ponto, tomar valores de $n = 1000, 2000, 3000, 4000$.

2. Considerar uma sequência de números gerada da seguinte forma:

$$x_n = a x_{n-1} (1 - x_{n-1}), \quad n = 1, 2, \dots, N$$

Considerar $N = 5000$, $x_0 = 0.1$ e diferentes valores de a entre 0 e 4 (p.e., $a = 1, 2, 3.8$ e 4). Calcular a média e a variância da sequência:

$$\bar{x} = \frac{1}{N} \sum_{i=0}^N x_i, \quad \sigma = \frac{1}{N-1} \sum_{i=0}^N (x_i - \bar{x})^2$$

Programar-lo na mão e usando funções de numpy já prontas. Comparar os resultados.

- No problema 2, plotar a sequência de valores obtida em cada caso considerando os diferentes valores de a pedidos. Fazer os gráficos usando legendas, labels, e outros atributos que achar interessante, para melhor ilustrar os resultados.
- Continuando com a sequência do problema 3, fazer um código que gera o diagrama de bifurcações, que é um gráfico que mostra os valores que assume a sequência, como função dos valores de a . O resultado deveria ser algo do tipo mostrado na figura ao lado, que no eixo horizontal tem os valores de a usados para gerar a sequência e no eixo vertical todos os possíveis valores que toma a sequência para o correspondente valor de a . Se sugere usar pontinhos bem pequenos para gerar o gráfico. Explicar os resultados se auxiliando com os gráficos

do exercício anterior.

5. Considerar uma rede de distribuição com a mostrada na figura ao lado. Esta pode ser um exemplo de uma rede elétrica ou hidráulica e é basicamente o que se chama um *grafo*). Notar que em geral ela estará caracterizada por um certo número de *nós* (ou uniões), um certo número de *arestas* e alguma informação sobre a conectividade entre pontos.

- ▶ Fazer uma estrutura de dados que sirva para descrever essa rede. Idealmente, a estrutura deveria incluir algum tipo de matriz ou *array* que indica como os nós e as arestas estão relacionados. Adicionalmente, a estrutura deve conter um array para descrever as coordenadas (x, y) de cada nó. Construir um exemplo e plotar a rede.
- ▶ Fazer uma função que deleta um nó e todas as arestas que emanam dele.
- ▶ Fazer uma outra função em python que permita apagar (ou *deletar*) uma aresta da rede. Notar que se a aresta possui um nó que não pertence a nenhuma outra aresta, esse nó também precisa ser deletado.
- ▶ Finalmente, fazer uma função que insere uma nova aresta na rede para conectar dois pontos já existentes.

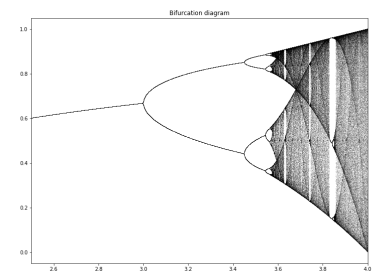


Figure 1.1: Diagrama de bifurcações.

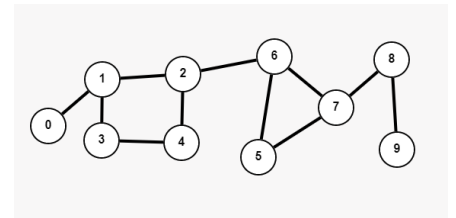


Figure 1.2: Exemplo de uma rede.

CÁLCULO DE REDES EM python: Resolução de Sistemas Lineares 1

2

2.1 Preludio

No presente capítulo iremos mostrar como se formula o problema de distribuição numa rede elétrica ou hidráulica, ou em geral, qualquer quantidade ou informação que se propaga por uma rede de maneira conservativa. De maneira geral temos os seguintes:

2.1	Preludio	5
2.2	Redes em python	12
2.3	Lista 2 de exercícios	15
2.4	Cálculos Monte Carlo em Redes	16
2.5	Atividade 1	18

Objetivos

- Entender a modelagem de distintos sistemas físicos tais como redes hidráulicas, elétricas ou inclusive treliças, do ponto de vista da Álgebra Linear;
- Aprender como resolver tais problemas de maneira sistemática no computador;

Há muitos sistemas físicos que podem ser modelizados como uma rede na qual se propaga uma certa quantidade física. Na tabela da sequência mostramos três exemplos clássicos que aparecem na engenharia:

Sistema	Nó	Variável nodal	Aresta	Variável de aresta	Lei de conservação
Elétrico	Conexão	Voltagem	Resistencia	Corrente	Carga elétrica
Hidráulico	União	Pressão	Cano	Vazão	Massa
Estrutural	Articulação	Deslocamento	Barra	Tensão	Momento

2.1.1 Exemplo de resolução de uma rede hidráulica

Nesta parte vamos trabalhar com redes hidráulicas. Para isto, vamos considerar o exemplo de uma rede simples mostrado na figura. Posteriormente, conseguiremos generalizar isto para resolver qualquer rede.

Para o exemplo particular temos: $n_c = 7$ canos e $n_v = 5$ nós:

O objetivo é determinar:

- As pressões em cada nó: $\mathbf{p} = [p_1 \dots p_5]^T$
- As vazões em cada cano: $\mathbf{Q} = [Q_1 \dots Q_7]^T$.

Conceitos necessários:

- Comportamento de um cano $\rightarrow$ Lei constitutiva

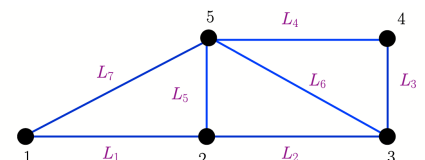


Figure 2.1: Exemplo de uma rede simples.

- Condições de acoplamento → **Conservação de massa e Continuidade**

Lei constitutiva: Comportamento de um cano

Para cada cano, $k = 1, \dots, n_c$, que conecta os nós i e j , vamos considerar a lei de perda de pressão em função da vazão:

$$\Delta p_k = p_i^k - p_j^k = \rho_k L_k Q_k$$

$$\Rightarrow Q_k = \frac{1}{\rho_k L_k} \Delta p_k = C_k (p_i^k - p_j^k)$$

- $C_k = \frac{1}{\rho_k L_k}$
- ρ_k é uma constante de resistência
- L_k é o comprimento de cada cano.
- A vazão é positiva quando vai de i a j .

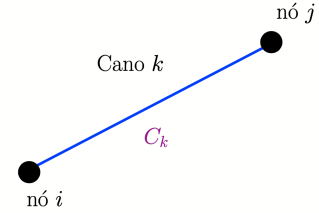


Figure 2.2: Um só cano.

Condições de acoplamento

- **Unicidade da pressão:** Para qualquer nó $i = 1, \dots, n_v$,

$$p_i^k = p_i$$

para todo nó k que possui o nó i .

- **Conservação da massa:**

$$\sum_{\forall k \text{ que possui o nó } i} Q_k^{(i)} = 0, \quad i = 1, 2, \dots, n_v$$

Desta forma, teremos uma equação por cada nó da rede. Para o caso particular do exemplo:

$$\begin{aligned} \text{Nó 1 : } \sum_k Q_k^{(1)} &= C_1(p_1 - p_2) + C_7(p_1 - p_5) &= 0 \\ \text{Nó 2 : } \sum_k Q_k^{(2)} &= C_1(p_2 - p_1) + C_5(p_2 - p_5) + C_2(p_2 - p_3) &= 0 \\ \text{Nó 3 : } \sum_k Q_k^{(3)} &= C_2(p_3 - p_2) + C_6(p_3 - p_5) + C_3(p_3 - p_4) &= 0 \\ \text{Nó 4 : } \sum_k Q_k^{(4)} &= C_3(p_4 - p_3) + C_4(p_4 - p_5) &= 0 \\ \text{Nó 5 : } \sum_k Q_k^{(5)} &= C_7(p_5 - p_1) + C_5(p_5 - p_2) + C_6(p_5 - p_3) + C_4(p_5 - p_4) &= 0 \end{aligned}$$

i.e., **5 equações e 5 incógnitas**. Agora vamos ordenar o sistema:

$$\begin{array}{rclclclclcl}
(C_1 + C_7)p_1 & - & C_1 p_2 & + & 0 p_3 & + & 0 p_4 & - & C_7 p_5 & = & 0 \\
-C_1 p_1 & + & (C_1 + C_2 + C_5)p_2 & - & C_2 p_3 & + & 0 p_4 & - & C_5 p_5 & = & 0 \\
0 p_1 & - & C_2 p_2 & + & (C_2 + C_3 + C_6)p_3 & - & C_3 p_4 & - & C_6 p_5 & = & 0 \\
0 p_1 & + & 0 p_2 & - & C_3 p_3 & + & (C_3 + C_4)p_4 & - & C_4 p_5 & = & 0 \\
-C_7 p_1 & - & C_5 p_2 & - & C_6 p_3 & - & C_4 p_4 & + & (C_4 + C_5 + C_6 + C_7)p_5 & = & 0
\end{array}$$

ou em forma matricial:

$$\begin{pmatrix} C_1 + C_7 & -C_1 & 0 & 0 & -C_7 \\ -C_1 & C_1 + C_2 + C_5 & -C_2 & 0 & -C_5 \\ 0 & -C_2 & C_2 + C_3 + C_6 & -C_3 & -C_6 \\ 0 & 0 & -C_3 & C_3 + C_4 & -C_4 \\ -C_7 & -C_5 & -C_6 & -C_4 & C_4 + C_5 + C_6 + C_7 \end{pmatrix} \cdot \begin{pmatrix} p_1 \\ p_2 \\ p_3 \\ p_4 \\ p_5 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

De forma geral, a matriz da rede descreve-se como

$$A_{ij} = \begin{cases} \sum_{k \text{ conectados com } i} C_k^{(i)} & \text{se } i = j \\ -C_r & \text{se } i \neq j, \text{ com } i \text{ conectado a } j \text{ por } C_r \\ 0 & \text{se não há conexão entre } i \text{ e } j \end{cases}$$

- ▶ A matriz é simétrica
- ▶ A soma dos elementos de qualquer linha/coluna é zero
- ▶ A matriz **não é invertível** → **A pressão está indeterminada**. Isto significa que não podemos resolver o sistema de equações até não eliminar essa indeterminação.
- ▶ Em geral, um nó estará conectado a um número relativamente pequeno de nós. Isto fará com que a matriz A possua muitos zeros

2.1.2 Conexão da rede

Agora, precisamos conectar a rede para resolver um problema que tenha algum sentido físico.

- ▶ Para remover a indeterminação se fixa o valor da pressão em algum ponto. Para o exemplo, vamos conectar o nó 3 à atmosfera, do qual podemos deduzir que:

$$p_3 = 0$$

Porém, agora teremos uma vazão Q_R para determinar.

- Também, vamos injetar uma vazão Q_B no nó 1, conectando uma bomba, e assim resolver um problema que tenha uma solução não trivial.

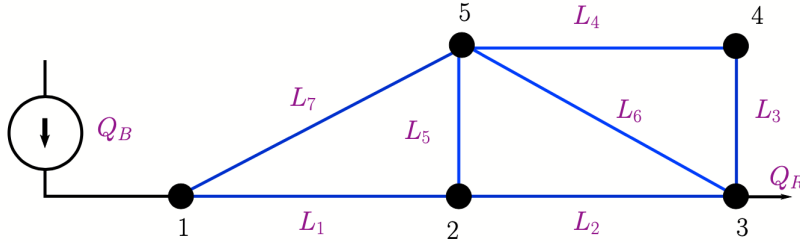


Figure 2.3: Rede conectada a uma bomba e descarregando na atmosfera.

Conexão da rede no nível algébrico

Precisamos incorporar no nível do sistema de equações a informação sobre a conexão da rede que acabamos de mencionar:

- Se estamos injetando no nó 1, então, a sua equação agora é:

$$\sum Q_r^{(1)} = (C_1 + C_7)p_1 - C_1 p_2 + 0 p_3 + 0 p_4 - C_7 p_5 = Q_B$$

- Similarmente, a equação para o nó 3 agora é:

$$\sum Q_r^{(3)} = 0 p_1 - C_2 p_2 + (C_2 + C_3 + C_6)p_3 - C_3 p_4 - C_6 p_5 = Q_R$$

em que Q_R é a **nova** incógnita. i.e., seria melhor colocar-la no lado esquerdo:

$$0 p_1 - C_2 p_2 + (C_2 + C_3 + C_6)p_3 - C_3 p_4 - C_6 p_5 - 1 Q_R = 0$$

- mas, se já sabemos que $p_3 = 0$, então, temos uma equação adicional:

$$0 p_1 + 0 p_2 + 1 p_3 + 0 p_4 + 0 p_5 = 0$$

No entanto, estamos interessados em achar as pressões, então, podemos **ignorar** a equação de balanço do nó 3 e ficar apenas com essa última equação, resultando o novo sistema de equações (ainda de 5×5):

$$\begin{pmatrix} C_1 + C_7 & -C_1 & 0 & 0 & -C_7 \\ -C_1 & C_1 + C_2 + C_5 & -C_2 & 0 & -C_5 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & -C_3 & C_3 + C_4 & -C_4 \\ -C_7 & -C_5 & -C_6 & -C_4 & C_4 + C_5 + C_6 + C_7 \end{pmatrix} \cdot \begin{pmatrix} p_1 \\ p_2 \\ p_3 \\ p_4 \\ p_5 \end{pmatrix} = \begin{pmatrix} Q_B \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Esse sistema matriz vetor, sim pode ser resolvido.

De forma geral podemos descrever a matriz $\tilde{A}$ e o vetor $\mathbf{b}$ do sistema como: Seja n_{atm} é o índice do nó em que a pressão é fixada ($p_{n_{atm}} = 0$) e n_B o índice do nó em que a bomba é conectada.

$$\tilde{A}_{ij} = \begin{cases} A_{ij} & \text{se } i \neq n_{atm} \\ 0 & \text{se } i = n_{atm} \text{ e } j \neq n_{atm} \\ 1 & \text{se } i = j = n_{atm} \end{cases}$$

$$b_i = \begin{cases} 0 & \text{se } i \neq n_{atm} \\ Q_B & \text{se } i = n_B \\ 0 & \text{se } i = n_{atm} \end{cases}$$

Ficando um sistema matriz-vetor do seguinte tipo:

$$\begin{pmatrix} A_{11} & A_{12} & \dots & A_{1n_{atm}} & \dots & A_{1n_B} & \dots & A_{1n_p} \\ A_{21} & A_{22} & \dots & A_{2n_{atm}} & \dots & A_{2n_B} & \dots & A_{2n_p} \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots & \dots & \vdots \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots & \dots & \vdots \\ 0 & 0 & \dots & 1 & \dots & 0 & \dots & 0 \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots & \dots & \vdots \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots & \dots & \vdots \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots & \dots & \vdots \\ A_{n_B1} & A_{n_B2} & \dots & A_{n_Bn_{atm}} & \dots & A_{n_Bn_B} & \dots & A_{n_Bn_p} \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots & \dots & \vdots \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots & \dots & \vdots \\ \vdots & \vdots & \dots & \vdots & \dots & \vdots & \dots & \vdots \\ A_{n_p1} & A_{n_p2} & \dots & A_{n_pn_{atm}} & \dots & A_{n_pn_B} & \dots & A_{n_pn_p} \end{pmatrix} \begin{pmatrix} p_1 \\ p_2 \\ \vdots \\ p_{n_{atm}} \\ \vdots \\ p_{n_B} \\ \vdots \\ p_{n_p} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ \vdots \\ Q_B \\ \vdots \\ 0 \end{pmatrix}$$

Exemplo numérico

Para colocar números concretos, consideremos uma vazão $Q_B = 3 \text{ m}^3/\text{s}$ e as seguintes propriedades para os canos da rede: Inserindo estes valores

Cano	ρ_k [bar s / m ⁴]	L_k [m]	C_k [bar s / m ³] ⁻¹
1	0.1	5	2
2	0.1	5	2
3	0.1	10	1
4	0.1	5	2
5	0.1	10	1
6	0.1	5	2
7	0.1	5	2

chegamos no sistema matriz-vetor (conferir):

$$\begin{pmatrix} 4 & -2 & 0 & 0 & -2 \\ -2 & 5 & -2 & 0 & -1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -2 \\ -2 & -1 & -2 & -2 & 7 \end{pmatrix} \cdot \begin{pmatrix} p_1 \\ p_2 \\ p_3 \\ p_4 \\ p_5 \end{pmatrix} = \begin{pmatrix} 3 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

E agora o que precisamos fazer é a resolução do sistema, o qual pode ser feito pelo chamado escalonamento da matriz:

$$\left(\begin{array}{ccccc|c} 4 & -2 & 0 & 0 & -2 & 3 \\ -2 & 5 & -2 & 0 & -1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 3 & -2 & 0 \\ -2 & -1 & -2 & -2 & 7 & 0 \end{array} \right) \xrightarrow{\ell_2 \leftarrow \ell_2 + \frac{1}{2}\ell_1} \left(\begin{array}{ccccc|c} 4 & -2 & 0 & 0 & -2 & 3 \\ 0 & 4 & -2 & 0 & -2 & \frac{3}{2} \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 3 & -2 & 0 \\ -2 & -1 & -2 & -2 & 7 & 0 \end{array} \right) \xrightarrow{\ell_5 \leftarrow \ell_5 + \frac{1}{2}\ell_1}$$

$$\left(\begin{array}{ccccc|c} 4 & -2 & 0 & 0 & -2 & 3 \\ 0 & 4 & -2 & 0 & -2 & \frac{3}{2} \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 3 & -2 & 0 \\ 0 & -2 & -2 & -2 & 6 & \frac{3}{2} \end{array} \right) \xrightarrow{\ell_5 \leftarrow \ell_5 + \frac{1}{2}\ell_2} \left(\begin{array}{ccccc|c} 4 & -2 & 0 & 0 & -2 & 3 \\ 0 & 4 & -2 & 0 & -2 & \frac{3}{2} \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 3 & -2 & 0 \\ 0 & 0 & -3 & -2 & 5 & \frac{9}{4} \end{array} \right) \xrightarrow{\ell_4 \leftarrow \ell_4 + \ell_3}$$

$$\left(\begin{array}{ccccc|c} 4 & -2 & 0 & 0 & -2 & 3 \\ 0 & 4 & -2 & 0 & -2 & \frac{3}{2} \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & -2 & 0 \\ 0 & 0 & -3 & -2 & 5 & \frac{9}{4} \end{array}\right) \xrightarrow{\ell_5 \leftarrow \ell_5 + 3\ell_3} \left(\begin{array}{ccccc|c} 4 & -2 & 0 & 0 & -2 & 3 \\ 0 & 4 & -2 & 0 & -2 & \frac{3}{2} \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & -2 & 0 \\ 0 & 0 & 0 & -2 & 5 & \frac{9}{4} \end{array}\right) \xrightarrow{\ell_5 \leftarrow \ell_5 + \frac{2}{3}\ell_4}$$

$$\left(\begin{array}{ccccc|c} 4 & -2 & 0 & 0 & -2 & 3 \\ 0 & 4 & -2 & 0 & -2 & \frac{3}{2} \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & -2 & 0 \\ 0 & 0 & 0 & 0 & \frac{11}{3} & \frac{9}{4} \end{array}\right) \xrightarrow{\ell_i \leftarrow \ell_i / a_{ii}} \left(\begin{array}{ccccc|c} 1 & -\frac{1}{2} & 0 & 0 & -\frac{1}{2} & \frac{3}{4} \\ 0 & 1 & -\frac{1}{2} & 0 & -\frac{1}{2} & \frac{3}{8} \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & -\frac{2}{3} & 0 \\ 0 & 0 & 0 & 0 & 1 & \frac{27}{44} \end{array}\right)$$

$$\left(\begin{array}{ccccc} 1 & -\frac{1}{2} & 0 & 0 & -\frac{1}{2} \\ 0 & 1 & -\frac{1}{2} & 0 & -\frac{1}{2} \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & -\frac{2}{3} \\ 0 & 0 & 0 & 0 & 1 \end{array}\right) \cdot \begin{pmatrix} p_1 \\ p_2 \\ p_3 \\ p_4 \\ p_5 \end{pmatrix} = \begin{pmatrix} \frac{3}{4} \\ \frac{3}{8} \\ 0 \\ 0 \\ \frac{27}{44} \end{pmatrix}$$

o que finalmente resulta:

$$\begin{aligned} 1p_1 - \frac{1}{2}p_2 + 0p_3 + 0p_4 - \frac{1}{2}p_5 &= \frac{3}{4} \rightarrow p_1 = \frac{123}{88} \\ 1p_2 - \frac{1}{2}p_3 + 0p_4 - \frac{1}{2}p_5 &= \frac{3}{8} \rightarrow p_2 = \frac{60}{88} \\ + 1p_3 + 0p_4 + 0p_5 &= 0 \rightarrow p_3 = 0 \\ + 1p_4 - \frac{2}{3}p_5 &= 0 \rightarrow p_4 = \frac{54}{132} \\ 1p_5 &= \frac{27}{44} \rightarrow p_5 = \frac{27}{44} \end{aligned}$$

Verificação dos resultados

Lembremos que a equação para o nó 3 era:

$$\sum Q_r^{(3)} = 0 p_1 - C_2 p_2 + (C_2 + C_3 + C_6) p_3 - C_3 p_4 - C_6 p_5 = Q_R$$

ou, colocando os valores da tabela

$$\sum Q_r^{(3)} = 0 - 2 p_2 + 5 p_3 - 1 p_4 - 2 p_5 = Q_R$$

e inserindo as pressões que acabamos de achar:

$$Q_R = \sum Q_r^{(3)} = 0 - 2 \frac{60}{88} + 0 - 1 \frac{54}{132} - 2 \frac{27}{44} = -3$$

ou seja, pelo nó 3 está saindo exatamente o que injectamos no nó 1.

2.2 Redes em python

A matriz de conectividades e montagem da matriz A

Uma forma “prática” de montar a matriz A , utiliza a chamada matriz de conectividades $\text{conec} \in \mathbb{N}^{n_c \times 2}$. Esta é uma estrutura muito conveniente para descrever a rede e é um ingrediente chave no método dos Elementos Finitos:

$$\text{conec} = \begin{pmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4 \\ 4 & 5 \\ 5 & 2 \\ 5 & 3 \\ 5 & 1 \end{pmatrix}$$

Olhando para a matriz **global** da rede, a ideia é ir acumulando nela, a matriz **local** associada a cada cano. Para o cano k , de condutividade C_k , teremos uma matriz de 2×2 , que chamaremos a matriz local do cano.

$$C_{loc_k} = \begin{pmatrix} C_k & -C_k \\ -C_k & C_k \end{pmatrix}$$

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \xrightarrow{\text{Cano 1}} \begin{pmatrix} C_1 & -C_1 & 0 & 0 & 0 \\ -C_1 & C_1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \xrightarrow{\text{Cano 2}}$$

$$\begin{pmatrix} C_1 & -C_1 & 0 & 0 & 0 \\ -C_1 & C_1 + C_2 & -C_2 & 0 & 0 \\ 0 & -C_2 & C_2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \xrightarrow{\text{Cano 3}} \begin{pmatrix} C_1 & -C_1 & 0 & 0 & 0 \\ -C_1 & C_1 + C_2 & -C_2 & 0 & 0 \\ 0 & -C_2 & C_2 + C_3 & -C_3 & 0 \\ 0 & 0 & -C_3 & C_3 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \dots$$

$$\dots \xrightarrow{\text{Cano 7}} \begin{pmatrix} C_1 + C_7 & -C_1 & 0 & 0 & -C_7 \\ -C_1 & C_1 + C_2 + C_5 & -C_2 & 0 & -C_5 \\ 0 & -C_2 & C_2 + C_3 + C_6 & -C_3 & -C_6 \\ 0 & 0 & -C_3 & C_3 + C_4 & -C_4 \\ -C_7 & -C_5 & -C_6 & -C_4 & C_4 + C_5 + C_6 + C_7 \end{pmatrix}$$

Para fazer tal montagem precisaremos programar uma função de python que automatize esse processo de montagem de matrizes locais, o que deveria ser algo do tipo:

```
def Assembly(conec, C):
    nv = ... # numero de nos
    nc = ... # numero de canos
    A = np.zeros(shape=(nv,nv))
    for k in range(nc):
        n1 = conec[k,0]
        n2 = conec[k,1]
        .
        .
    return A
```

em que C é o vetor que possui os valores das conductâncias dos canos da rede. Os detalhes ficarão como exercício para a lista 2.

Montagem do sistema final e Resolução

Uma vez que temos a matriz A do sistema, precisamos montar a matriz $\tilde{A}$ e o vetor de lado direito que é forma o sistema de equações definitivo que precisa ser resolvido para determinar as pressões nos nós da rede, que é o que nos interessa em concreto. Para isto, precisaremos programar uma função de python que modifique a linha da matriz na posição $natm$ daquele nó que foi conectado à atmosfera e também cria um vetor de lado direito que incorpora a informação da vazão injetada pela bomba QB na posição nB . A função deveria ser algo do tipo¹

```
def SolveNetwork(conec, C, natm, nB, QB):
    Atilde = Assembly( ... )
    Atilde[natm, :] = ...
    .
    .
    b = np.zeros( ... )
    .
    pressure = np.linalg.solve(Atilde, ... )
    return pressure
```

1: A função `np.linalg.solve` essencialmente faz o escalonamento da matriz para encontrar a solução.

Os detalhes ficarão como exercício para a lista 2.

Post-processo da solução

Uma vez resolvido o problema, estamos interessados em analisar a solução, para extrair informações de interesse do cálculo:

- ▶ Cálculo das vazões nos canos;
- ▶ Cálculo da potência consumida pela bomba;
- ▶ Visualização e plotagem da solução.

Para o primeiro ponto, uma forma elegante de fazer o cálculo é usar a seguinte fórmula:

$$\mathbf{Q} = \mathbf{K} \mathbf{D} \mathbf{p}$$

em que $\mathbf{K} \in \mathbb{R}^{n_c \times n_c}$ é a matriz diagonal com as conductâncias dos canos definida por

$$\mathbf{K}_{ij} = \begin{cases} C_i & \text{se } i = j \\ 0 & \text{no resto} \end{cases}$$

e a matriz $\mathbf{D} \in \mathbb{R}^{n_c \times n_v}$ é a matriz definida por:

$$\mathbf{D}_{kj} = \begin{cases} 1 & \text{se } j = \text{conec}[k, 0] \\ -1 & \text{se } j = \text{conec}[k, 1] \\ 0 & \text{no resto} \end{cases}$$

Para o segundo ponto, pode-se provar que a potência consumida pela bomba para fazer circular o fluido é dada por:

$$W = \mathbf{p}^T (\mathbf{D}^T \mathbf{K} \mathbf{D}) \mathbf{p}$$

isto surge do fato de que a potência consumida pela bomba tem que ser igual a energia dissipada nos canos da rede, i.e.,

$$W = \sum_{k=1}^{n_e} Q_k \Delta p_k$$

em que Q_k é a vazão pelo cano k e Δp_k é a diferença de pressão nos extremos do cano k . Mas, se colocarmos as vazões e as diferenças de pressão em vetores, podemos escrever

$$W = \mathbf{Q} \cdot \Delta \mathbf{p} = \mathbf{Q}^T \Delta \mathbf{p}$$

i.e., W é o igual ao produto escalar desses vetores. Agora, notemos que $\Delta \mathbf{p} = \mathbf{D} \mathbf{p}$ e $\mathbf{Q} = \mathbf{K} \mathbf{D} \mathbf{p}$, então

$$W = \mathbf{Q}^T \Delta \mathbf{p} = (\mathbf{K} \mathbf{D} \mathbf{p})^T \Delta \mathbf{p} = (\mathbf{p}^T \mathbf{D}^T \mathbf{K}^T) (\mathbf{D} \mathbf{p}) = \mathbf{p}^T (\mathbf{D}^T \mathbf{K} \mathbf{D}) \mathbf{p}$$

pois o produto de matrizes é **associativo** e $\mathbf{K} = \mathbf{K}^T$, por ser ela uma matriz diagonal².

Finalmente, para visualizar a solução, na lista de exercícios iremos disponibilizar algumas funções de python que permitirão mostrar a distribuição de pressões em certo tipo de redes como a mostrada na figura.

Aviso importante

O material de estudo para este capítulo serão as notas apresentadas no presente capítulo, os jupyter notebooks disponibilizados pelo professor, assim como consultas dirigidas ao professor em forma presencial ou por e-mail (rfausas@icmc.usp.br).

2.3 Lista 2 de exercícios

A lista de exercícios a ser apresentada em avaliação oral por sorteio é dada na sequência:

Exercícios

1. Completar a função de python que faz a montagem da matriz A . Testar na rede que tem sido usada como exemplo.
2. Completar a função de python que cria a matriz $\tilde{A}$ e vetor de lado direito do sistema b de equações definitivo a ser resolvido, resolve o sistema e retorna o vetor de pressões com a solução. Testar na rede que tem sido usada como exemplo. Conferir a resposta com a obtida anteriormente.

2: Notar que o resultado é um número, i.e., uma matriz de 1×1 , pois a potência que tem unidades de Energia por unidade de tempo (p.e., [J/s]) é de fato uma quantidade escalar.

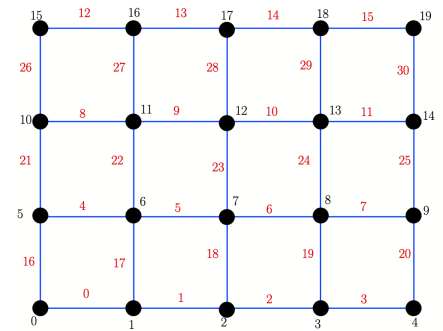


Figure 2.4: Exemplo de uma rede de tipo grade possuído $m = 5$ nós na horizontal e $n = 4$ nós na vertical (i.e., $n_v = m n = 20$ nós). Notar que a numeração de nós e arestas já é feita começando em 0.

3. Fazer uma função de python que calcula o vetor de vazões na rede segundo o explicado anteriormente. Explicar como é que o cálculo das matrizes **K** e **D** leva a determinar as vazões.
4. Incluir o cálculo da potência na função que foi programada no exercício anterior.
5. Considerar a rede hidráulica de tipo grade gerada com a função que foi disponibilizada no jupyter notebook. Montar um exemplo de resolução no qual a rede possui 10×12 nós, uma vazão $Q_B = 3 \text{ m}^3/\text{s}$ é injetada no nó do canto inferior esquerdo e a descarga à atmosfera é feita pelo nó do canto superior direito. Aplique todas as funções desenvolvidas até agora e visualize a solução usando a função `PlotPressure` disponibilizada no jupyter notebook. Mudar o ponto de descarga e comparar com a solução anterior.
6. Considerar redes hidráulicas de tipo grade de tamanho variável $m \times n$ (p.e., 10×10 , 20×20 , 40×40 , 80×80 , etc.) Fazer um código que resolve o sistema para encontrar a solução e mede o tempo de cálculo envolvido. Plotar em escala loglog o tempo como função do número de incógnitas $n_v = m \cdot n$.

2.4 Cálculos Monte Carlo em Redes

Objetivos

- Realizar cálculos estocásticos para estimar a probabilidade de certos eventos em sistemas complexos, tais como uma rede hidráulica composta de centenas ou milhares de arestas.
- Estes cálculos estocásticos são a base dos chamados métodos Monte Carlo, que são uma classe de métodos computacionais baseados em amostragens aleatórias.
- Estes métodos são muito poderosos e flexíveis para resolver problemas complexos. Muitas vezes são a única forma de resolver-los.

Vejam um exemplo de um problema complexo que poderíamos resolver usando estes métodos:

Análise de risco

- Um bairro conta com uma rede hidráulica conhecida (ver figura Figure 2.5), alimentada por uma bomba que injeta uma vazão $Q_B = 10 \text{ m}^3/\text{s}$ no nó do canto inferior esquerdo, i.e., o nó $n_B = 0$ das redes que geramos usando a função `GeraRede(...)`
- No nó $n_{atm} = n_v - 1$ é fixada uma pressão nula.
- A rede possui dois tipos de canos: **grossos**, com “condutância” $C = 20$, e os **finos**, com $C = 2$.
- Anualmente, os canos **finos** (apenas eles) ficam obstruídos com probabilidade p_O . Quando obstruídos, seu valor de “condutância” se reduz a $\text{Centup} = 0.2$, ou seja, 10 vezes menor do que o valor

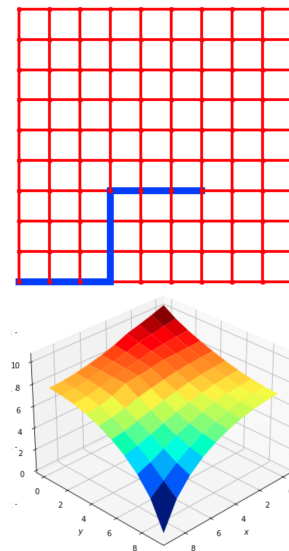


Figure 2.5: Rede nominal que será usada para realizar a análise de risco.

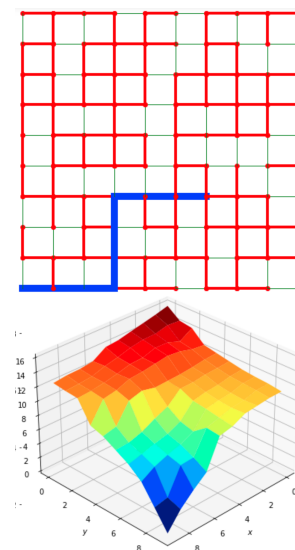


Figure 2.6: Exemplo de uma rede com canos entupido que será usada para realizar a análise de risco.

nominal (ver exemplo de uma rede com obstruções na figura Figure 2.6).

Pergunta

Qual é a probabilidade de que, passado um ano, a pressão máxima na rede seja maior do que 12?

Mas, vejamos que acontece neste problema:

- ▶ Seja M o número de canos finos, todos distintos por ocuparem lugares distintos na rede.
- ▶ Existem 2^M casos diferentes. Então, supondo, por exemplo, $M = 100$ teremos $2^{100} = 1.27 \times 10^{30}$ casos.
- ▶ Cada caso que deve ser analisado corresponde a um vetor de condutâncias da rede diferente.
- ▶ Para cada caso, podemos calcular deterministicamente todas as pressões, em particular a máxima:

```
Cnew = RandomFailFinos(C, p0, Centup)
pressure = SolveNetwork(conec, Cnew, nB, QB, natm)
pmax = np.max(pressure)
```

em que a função `RandomFailFinos` gera uma instancia de possíveis canos entupidos na rede.

- ▶ Então, podemos avaliar a probabilidade como:

$$\text{Prob} = \frac{\# \text{ num. de casos que aconteceu o evento}}{\# \text{ num. total de possíveis casos}}$$

- ▶ Se formos calcular todos os possíveis casos surge uma **dificuldade**:

Aviso importante

Se o cálculo de um caso particular demora **1 nanosegundo**, para analisar todos demoramos 4×10^{13} anos.

Para entender a ideia por trás dos métodos Monte Carlo (MC), e como que eles podem nos auxiliar para responder a pergunta anterior, vamos fazer um exemplo simples:

Cálculo do número π

Vamos estimar el valor do número π da seguinte forma: Se consideramos a região quadrada $[-1, 1] \times [-1, 1]$ e um círculo inscrito, a razão das áreas entre estes é $\pi/4$ (ver figura).

Então, podemos propor um cálculo estocástico que consistirá em jogar pontos dentro dessa região quadrada. Alguns desses pontos irão cair dentro do círculo e outros fora. Como é de se esperar, a razão entre a quantidade de pontos que cai dentro e a quantidade total de pontos jogados, deveria tender justamente a razão entre as áreas de círculo e do quadrado ($\pi/4$).

Se denotarmos por N_c e N ao número de pontos dentro do círculo e ao número total de pontos, respectivamente:

$$\text{Prob} = \frac{\pi}{4} = \lim_{N \rightarrow \infty} \frac{\text{\#num. pontos no círculo}}{\text{\#num. total de pontos}} = \lim_{N \rightarrow \infty} \frac{N_c}{N}$$

Isto leva ao seguinte roteiro que precisamos executar:

1. Gerar (x_i, y_i) , formado por dois números x_i e y_i entre -1 e 1 , independentes e com probabilidade uniforme.
2. Calcular θ_i , definida como igual a 1 se $x_i^2 + y_i^2 < 1$ e igual a 0 se não. Seja $\Theta = \{\theta_i\}$ a sequência gerada.
3. Tirar a média

$$\hat{\theta} = \frac{1}{N} \sum_{i=1}^N \theta_i$$

o qual se materializa no seguinte código de python:

```
# Computation of pi by stochastic method

N = 10000 # Number of realizations
Nc = 0
for i in range(N):
    x = -1.0 + 2.0*np.random.rand()
    y = -1.0 + 2.0*np.random.rand()
    if (x**2 + y**2 < 1.0):
        Nc = Nc + 1;

Prob = Nc / N
print('The estimate for pi is:', 4*Prob)
```

Os resultados para diferente número de realizações se mostram na figura ao lado e na tabela seguinte para $4 \times p$ (o que deveria tender a π)³:

#	$N = 10^3$	$N = 10^4$	$N = 10^5$	$N = 10^6$	$N = 10^7$
1	3.0800	3.1492	3.1450	3.1418	3.1405
2	3.2320	3.1340	3.1396	3.1400	--
3	3.1240	3.1412	3.1469	3.1376	--
4	3.0800	3.1660	3.1488	3.1406	--

Agora que entendemos como funciona um método MC vamos aplicar ele para fazer a análise de risco em redes hidráulicas, que é o tema desenvolvido na Atividade 1.

2.5 Atividade 1

A primeira atividade a ser desenvolvida em grupo e com relatório que será entregue em data a ser definida, segue na sequência:

Exercícios

1. Fazer uma função que pega as condutâncias da rede e gera uma lista com as novas condutâncias dos valores individuais dos canos que foram entupidos numa determinada realização.

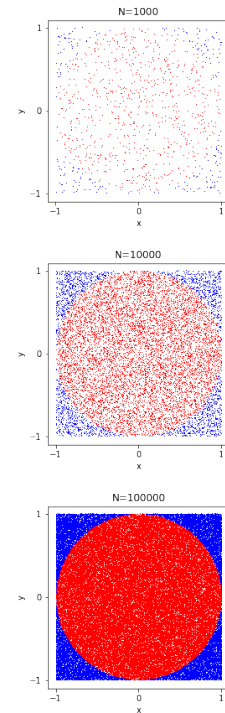


Figure 2.7: Exemplo do cálculo MC.

3: Um dos pontos a destacar é que estamos usando uma distribuição uniforme de probabilidades (`np.random.rand()`) para gerar os pontos randômicos, i.e., a probabilidade é a mesma para qualquer número em $(0, 1)$ e os pontos gerados são "independentes" um dos outros. Isto é um ingrediente essencial do método MC.

Considerar uma estrutura do tipo:

```
def RandomFailFinos(C, p0, Centup):
    Cnew = np.copy(C)
    nc = ...
    for k in range(nc):
        x = np.random.rand()
        if( ..... ):
            Cnew[k] = ...

    return Cnew
```

2. Realizar o cálculo de probabilidade Prob de que, passado um ano, a pressão máxima na rede seja maior do que 12?

Para isto:

- ▶ Seguir um roteiro de cálculo similar ao que foi feito no exemplo de cálculo do número π ;
- ▶ Realizar entre 5000 e 10000 realizações para cada caso;
- ▶ Considerar diferentes valores de p_0 e fazer uma tabela mostrando Prob como função de p_0 ;
- ▶ Plotar a evolução dessa probabilidade como função do número de realizações;
- ▶ Tirar conclusões.

2.5.1 Exercícios opcionais

Na sequência tem um exercício extra que é opcional. Quem quiser, pode apresentar ele para o professor de maneira individual, valendo como uma das avaliações orais.

BONUS EXTRA

1. Considerar uma rede com os seguintes parâmetros:

- ▶ $n = 8$
- ▶ $m = 9$
- ▶ $QB = 3$
- ▶ $natm = n*m - 1$
- ▶ $nB = 0$
- ▶ As condutâncias dependem de um parâmetro x . Considerar os casos:

$$\begin{aligned} CH &= 2.3 + 0.1(x-1)^2, \\ CV &= 1.8 + 0.2(x-1)^2 \end{aligned}$$

e

$$\begin{aligned} CH &= 2.3 + 10e^{-(x-5)^2}, \\ CV &= 1.8 + 10e^{-(x-5)^2} \end{aligned}$$

A ideia é plotar em cada caso, a potência dissipada como função do parâmetro x e determinar visualmente para que valor de x a potencia é igual a 6. Barrer valores de x num intervalo

interessante. Aplicar as funções desenvolvidas nos exercícios anteriores.

PROBLEMA DE CALOR e ITERAÇÕES NA ENGENHARIA: Resolução de Sistemas Lineares 2

3

3.1 Preludio

O conceito de processo iterativo é muito frequente na engenharia para resolver problemas complexos, nos quais não é possível obter uma solução fechada dos problemas numa primeira tentativa. Ele é usado, tanto nas etapas de desenho como de cálculo via modelos físicos. Neste capítulo iremos estudar o problema de **transferência de calor por condução** em materiais. Para isto, vamos introduzir métodos e algoritmos para resolver o problema de encontrar a distribuição de temperaturas num material, tal como é mostrado na figura ao lado. Neste capítulo temos os seguintes:

Objetivos

- ▶ Entender a modelagem do problema de transferência de calor por condução
- ▶ Apresentar métodos iterativos **sem matriz** e **com matriz**;
- ▶ Implementar os cálculos em python;
- ▶ Usar bibliotecas de métodos iterativos disponíveis em scipy

3.1 Preludio	21
3.2 Transferência de Calor por Condução	21
3.3 Métodos iterativos	24
3.4 Lista 3 de exercícios	27

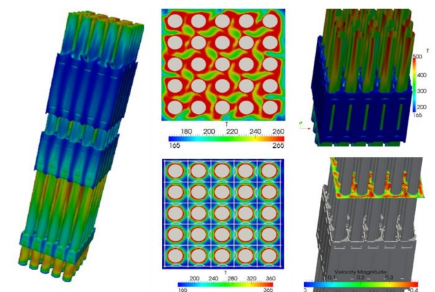


Figure 3.1: Exemplo da distribuição de temperaturas num elemento combustível de uma central nuclear sendo refrigerado por um fluido.

3.2 Transferência de Calor por Condução

Como no problema das redes hidráulicas precisamos introduzir alguns conceitos para poder formular o problema:

- ▶ **Lei constitutiva:** a qual relaciona o fluxo de energia, na forma de calor, neste caso devido a uma variação ou diferença de temperatura:

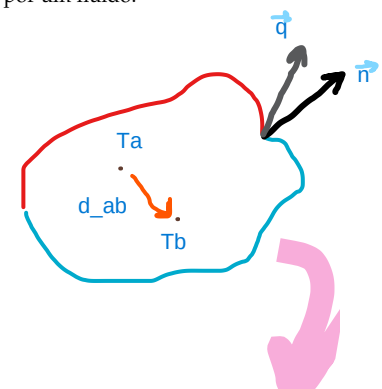
$$\mathbf{q}(x, y, t) = -k \nabla T(x, y, t)$$

em que (x, y, t) denota o ponto espacial e o tempo t , k é a chamada **condutividade térmica**, que é uma propriedade do material, que assume-se conhecida, T é a **distribuição de temperaturas**, a qual pretendemos encontrar e o operador ∇ denota o **gradiente**, que no caso bidimensional é¹:

$$\nabla T(x, y, t) = \begin{bmatrix} \frac{\partial T}{\partial x} \\ \frac{\partial T}{\partial y} \end{bmatrix}$$

- ▶ Uma lei de balanço de energia:

$$\underbrace{\frac{d}{dt} \iiint_V \rho c T dv}_{\text{Taxa de variação da energia interna}} = \underbrace{\iint_S \mathbf{q} \cdot \mathbf{n} ds}_{\text{Calor que ingresa ou sai pela superfície}} + \underbrace{\iiint_V f dv}_{\text{Calor gerado internamente no volume}}$$



1: Na prática, o gradiente da função será aproximado por uma expressão do tipo:

$$\nabla T(x, y, t) \approx \frac{T_A - T_B}{|d_{AB}|} \frac{\vec{d}_{AB}}{|d_{AB}|}$$

, i.e., a partir da diferença de temperatura entre dois pontos próximos A e B, sendo d_{AB} a distância entre eles.



em que c é a capacidade calorífica do material, ρ é a densidade, f é a fonte geradora de calor (p.e., uma reação nuclear, uma reação química, uma corrente elétrica) e $\mathbf{n}$ é o vetor normal unitário exterior à superfície do sólido.

Neste capítulo iremos supor que o problema é estacionário, i.e., a temperatura não mudará com o tempo, e por tanto podemos negligenciar o termo do lado esquerdo, resultando no seguinte problema matemático que precisamos resolver para encontrar a distribuição de temperaturas $T(x, y)^2$:

$$\oint_S k \nabla T \cdot \mathbf{n} \, ds = \iiint_V f \, dv \quad (3.1)$$



2: A equação 3.1 deve valer para qualquer volume arbitrários V limitado por um contorno ou superfície S .

Para termos uma ideia, o coeficiente de condutividade térmica, denotado por k é uma propriedade que pode ter valores bem diferentes dependendo do material:

Material	Condutividade k [W · m ⁻¹ · K ⁻¹]
Aluminio	230
Aço	20
Concreto	1.5
Plástico isolante	0.03
Ar	0.02

Em geral, k assume-se constante, porém também pode depender da própria temperatura ou outras variáveis, mas isso torna o problema não linear e muito mais difícil de se resolver. Neste capítulo iremos nos restringir ao caso linear e estudaremos como resolver o sistema de equações que resulta de aplicar a lei de conservação introduzida acima.

Formulação do sistema de equações

Para poder resolver o problema, em geral, precisamos efetuar o que se conhece como discretização. Isto consiste em dividir o domínio computacional em parcelas que são chamadas células³. Isto é ilustrado na figura 3.2, onde um domínio computacional quadrado (figura à esquerda) e a sua discretização em células quadradas (figura à direita) são mostrados. Na figura também indicamos as temperaturas que iremos impor nas bordas do domínio (no topo T_T , embaixo T_B , na esquerda T_L e na direita T_R), as quais devem ser conhecidas para poder efetivamente resolver o problema.

3: Asumiremos que todas as células são quadradas de lado h , então, se $h \rightarrow 0$, quanto maior seja o número de células, mais acurada será a solução numérica obtida.

essas bordas equivalem a paredes

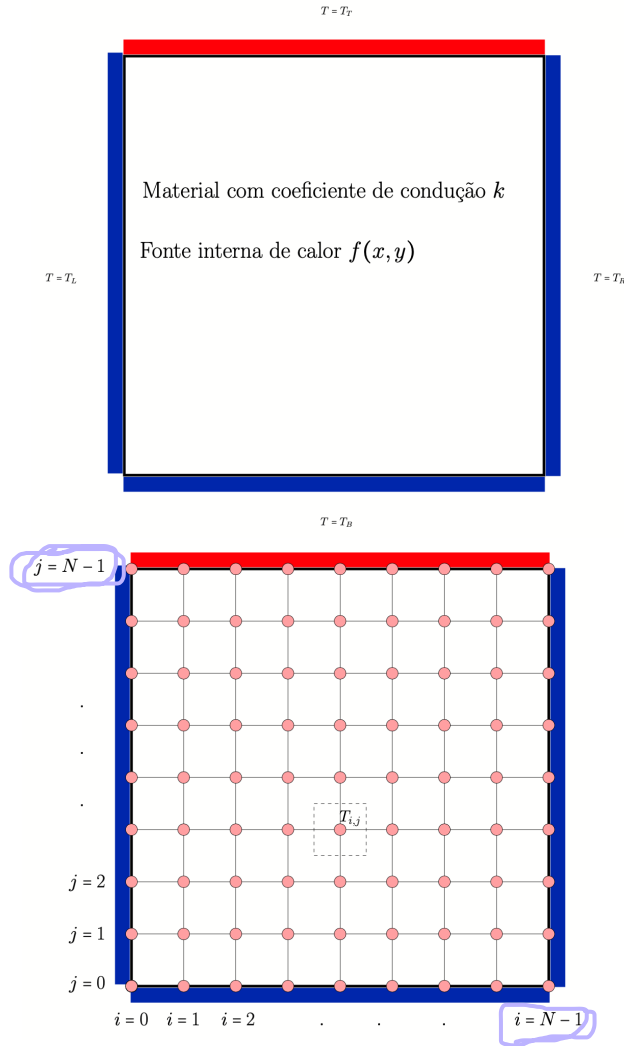


Figure 3.2: Volume e partição em células onde será resolvido o problema de condução de calor.

Olhando para a figura, vemos que temos uma grade quadrada, cujos vértices, indicados pelos pontos rosas, têm sido indexados pelos índices (i, j) . Ao redor de cada ponto rosa temos desenhado com linha tracejada, um volume⁴ V arbitrário de forma quadrada. A ideia é encontrar as temperaturas $T_{i,j} \forall i, j$.

Para formular o problema, temos que escrever a lei de balanço (3.1) para cada possível volume da discretização. Para isto considerar a figura Figure 3.3. Podemos calcular facilmente o fluxo de calor passando por cada face do contorno do quadrado. Lembrando que a h é a distância entre pontos, temos que uma boa aproximação dos fluxos é dada por:

$$\begin{aligned} q_e &\approx -k \frac{T_{i+1,j} - T_{i,j}}{h} \mathbf{e}_x \\ q_w &\approx -k \frac{T_{i,j} - T_{i-1,j}}{h} \mathbf{e}_x \\ q_n &\approx -k \frac{T_{i,j+1} - T_{i,j}}{h} \mathbf{e}_y \\ q_s &\approx -k \frac{T_{i,j} - T_{i,j-1}}{h} \mathbf{e}_y \end{aligned}$$

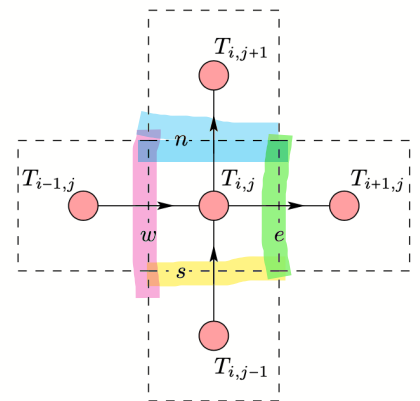


Figure 3.3: Célula (i, j) e seus 4 vizinhos, intercambiando calor a través da sua supefícies, limitada pelos contornos n (north), s (south), w (west) e e (east).

$$-k * \frac{(T_{\text{chegada}} - T_{\text{saída}})}{\text{dist entre pontos}} * \text{vetor unitário exterior da superf}$$

estimativa do gradiente $\hat{\mathbf{i}}$

Agora, somando todos os fluxos e calculando o termo de geração de calor:

$$\begin{aligned} \oint_S k \nabla T \cdot \mathbf{n} \, ds &\approx \sum_{\text{Fases do contorno de } V} \mathbf{q}_f \cdot \mathbf{n}_f s_f \approx \text{fluxo} \cdot \text{vetor} \cdot \text{tam da face} \\ &\approx h \left[\frac{k}{h} (T_{i,j} - T_{i+1,j}) + \frac{k}{h} (T_{i,j} - T_{i-1,j}) + \frac{k}{h} (T_{i,j} - T_{i,j+1}) + \frac{k}{h} (T_{i,j} - T_{i,j-1}) \right] = \iiint_V f \, dv \approx h^2 f \end{aligned}$$

Observação

Para o caso em que $f = 0$, a distribuição interna de temperaturas é apenas uma consequência das temperaturas que estivermos impondo nas bordas. Ainda, nesse caso, sendo que lado direito da equação fica igual a zero, teremos que:

$$-T_{i+1,j} - T_{i-1,j} + 4T_{i,j} - T_{i,j+1} - T_{i,j-1} = 0$$

O que significa que a temperatura num ponto é a **média** das temperaturas nos seus vizinhos:

$$T_{i,j} = \frac{T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1}}{4}$$

Mas, não conhecemos as temperaturas! Isso é justamente o que queremos calcular.

3.3 Métodos iterativos

A fórmula anterior motiva o seguinte método iterativo para encontrar as temperaturas para todos os pontos no interior do material:

Método iterativo de Jacobi

O método consiste em **atualizar a temperatura em todos os pontos internos, com base na temperatura calculada numa iteração anterior, partindo de algum chute inicial arbitrário**. O pseudo-código fica:

- ▶ Dado um chute inicial $T_{i,j}^{(0)}$, $i, j = 1, \dots, N-2$,
- ▶ Para $k = 1, \dots, MAX_IT$

$$T_{i,j}^{(k)} = \frac{T_{i+1,j}^{(k-1)} + T_{i-1,j}^{(k-1)} + T_{i,j+1}^{(k-1)} + T_{i,j-1}^{(k-1)}}{4}$$

A implementação resulta:

```

# Metodo de Jacobi

N = 11 # Numero de pontos em cada direcao

Told = np.zeros(shape=(N,N))
Tnew = np.zeros(shape=(N,N))

# Temperaturas nas bordas
Told[0, :] = 0.0 # TL
Told[N-1, :] = 0.0 # TR
Told[:, 0] = 0.0 # TB
Told[:, N-1] = 20.0 # TT

# Loop de iteracoes
Nmax = 10000
Tnew = Told.copy()
for iter in range(Nmax):
    Tnew[1:N-1, 1:N-1] = 0.25*(Told[2:N, 1:N-1] + Told[0:N-2, 1:N-1]
    + \
                                Told[1:N-1, 2:N] + Told[1:N-1, 0:N-2])

    error = np.linalg.norm(Tnew-Told)
    if(error < 1.0e-8):
        print('Converged in %d iterations\n' %(iter))
        break ;

    Told = Tnew.copy()

```

Notar que nas paredes $T_B = T_L = T_R = 0$ e na parede superior $T_T = 20$. Na figura Figure 3.4

Método iterativo de Gauss-Seidel

Neste método, a diferença é que **atualizamos a temperatura em todos os pontos internos**, não apenas com a temperatura da iteração anterior, mas, quando possível, **com a temperatura atualizada dos pontos pelos quais já temos passado, partindo de algum chute inicial arbitrário**. O pseudo-código fica:

- ▶ Dado um chute inicial $T_{i,j}^{(0)}$, $i, j = 1, \dots, N-2$,
- ▶ Para $k = 1, \dots, MAX_IT$

$$T_{i,j}^{(k)} = \frac{T_{i+1,j}^{(k-1)} + T_{i-1,j}^{(k)} + T_{i,j+1}^{(k-1)} + T_{i,j-1}^{(k)}}{4}$$

A implementação resulta:

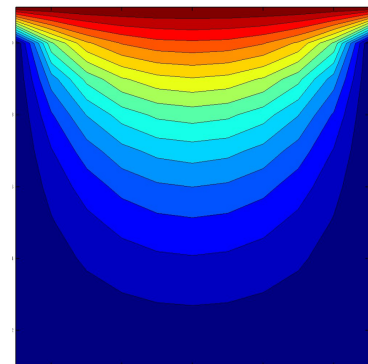
```

# Metodo de Gauss-Seidel

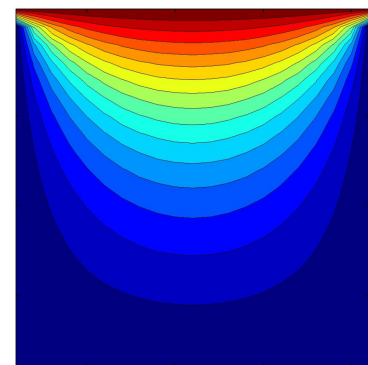
N = 11 # Numero de pontos em cada direcao

Told = np.zeros(shape=(N,N))
Tnew = np.zeros(shape=(N,N))

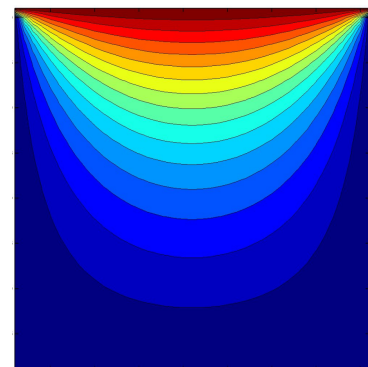
```



10 × 10



20 × 20



40 × 40

Figure 3.4: Distribuição de temperaturas para obtidas pelo método iterativo de Jacobi para diferentes níveis de discretização.

```
# Temperaturas nas bordas
Told[0, :] = 0.0 # TL
Told[N-1, :] = 0.0 # TR
Told[:, 0] = 0.0 # TB
Told[:, N-1] = 20.0 # TT

# Loop de iteracoes
Nmax = 10000
Tnew = Told.copy()
for iter in range(Nmax):
    for i in range(1, N-1):
        for j in range(1, N-1):
            Tnew[i, j] = 0.25*(Told[i+1, j] + Tnew[i-1, j] + \
                               Told[i, j+1] + Tnew[i, j-1])

    error = np.linalg.norm(Tnew-Told)
    if(error < 1.0e-8):
        print('Converged in %d iterations\n' %(iter))
        break ;

Told = Tnew.copy()
```

Aviso importante

O material de estudo para este capítulo serão as notas apresentadas no presente capítulo, os jupyter notebooks disponibilizados pelo professor, assim como consultas dirigidas ao professor em forma presencial ou por e-mail (rfausas@icmc.usp.br).

3.4 Lista 3 de exercícios

A lista de exercícios a ser apresentada em avaliação oral por sorteio é dada na sequência:

Exercícios

1.

Alphabetical Index

preface, ii